

class rixPDO.php

wrapper around PDO for mySQL (v1.75)

Methods:

constructor

rixPDO(\$db, \$user, \$password, \$host, \$logfile, \$errorHandling, &\$errorVar, \$errorVarKey, \$flags)

Parameters:

\$db	name of the database
\$user	database login
\$password	database password
\$host	url of database server if null or omitted, localhost will be used
\$logfile	path & filename of logfile for logging failed database operations if null or omitted, no log file will be written
\$errorHandling	0: rixPDO only reports errors, but does not handle them (default) 1: the error message is written into the variable <code>\$errorVar</code> sent by reference (next paramater) and then the <code>exit()</code> command is executed automatically 2: when an error occurs an <code>exit(\$errorMessage)</code> command is executed so that the error message is sent to the output buffer
&\$errorVar	variable given by reference for rixPDO to report the error message back when using errorHandling mode 1
\$errorVarKey	if <code>&\$errorVar</code> is an array, this string indicates under which key the error is written to the variable; if omitted the error message will write directly into <code>&\$errorVar</code> .
\$flags	an array with flags: • <code>emulatePrepares</code> (default: <code>false</code>) tells PDO driver to emulate prepare statements rather than do them natively in mySQL. <code>false</code> allows to fetch native datatypes (e.g. integers) <code>true</code> allows to get timestamps with decimal digits • <code>timeout</code> (default: <code>10</code>) sets the time in seconds before failing with a timeout when trying to connect to a database.

method execute

Executes the given query and returns the number of affected rows and the last autoincrement id.

Parameters:

\$query	the query to execute
---------	----------------------

Returns:

Results set as described in `results` method. Note that the `data` field of the results set is not created by this method, which purely executes and reports back `error`, `rows` and `id` fields. Use one of the fetch methods for `SELECT` queries.

method prepare

Send a query to the server to prepare.

Parameters:

\$query	query to prepare with '?' placeholders for the parameters or with named parameters (e.g. <code>:id</code>)
---------	---

Returns:

TRUE if it succeeded. Otherwise it returns a results set as described in **results** method describing the error.

method executePrepared

Executes a previously prepared statement.

Parameters:

\$parameters the parameters to bind to the prepared statement, as an array
if the query was defined with named parameters, the keys of the array represent the names

Returns:

Results set as described in **results** method. Note that the **data** field of the results set is not created by this method, which purely executes and reports back **error**, **rows** and **id** fields. Use one of the fetch methods for **SELECT** queries.

method fetchPrepared

Fetches data using a previously prepared statement.

Parameters:

\$parameters the parameters to bind to the prepared statement, as an array
if the query was defined with named parameters, the keys of the array represent the names

\$format defines the format under which the data should be returned (see below)

\$key filename that should be used as key for the rows (should logically be a unique field)
if not specified or if no such key exists in the result set, the rows will simply be numbered
for the formats that do not use multiple rows, this parameter is ignored

\$secondaryKey if a primary key has been defined, the secondary key can be used to create a more complex hierarchy (the two keys together must form a unique combined value)

\$aggregateRows boolean; if TRUE the row will not be added directly under a key but every row with the same key will be added to a subarray under the given key

Possible formats:

'table' data is returned as an array of rows, every row is indexed by column names

'tablename' data is returned as an array of rows, every row is indexed by numbers
if keys are specified they need to be integers (e.g. 0 for the first field)

'row' first row from result set is returned, indexed by column names
the key parameters are not applicable for this format

'rownum' first row from result set is returned, indexed by numbers
the key parameters are not applicable for this format

'column' the first field of each row is returned as an array indexed by numbers
if the first field is chosen as key, the data returned will be taken from the second field instead of the first, and similarly the data will be served from the third field if the first two fields are used as primary and secondary keys

'value' the first field from the first row will be returned
the key parameters are not applicable for this format

Returns:

Results set as described in **results** method.

methods `fetchTable` / `fetchTableNum` / `fetchRow` / `fetchRowNum` / `fetchColumn` / `fetchValue`

These are convenience methods for preparing a query, fetching and delivering results in one command. The name of the method decides which format the data should be delivered in (see `fetchPrepared` method for a description of all possible formats).

Parameters

<code>\$query</code>	query to prepare with '?' placeholders for the parameters or with named parameters (e.g. ':id')
<code>\$parameters</code>	the parameters to bind to the prepared statement, as an array if the query was defined with named parameters, the keys of the array represent the names
<code>\$key</code>	check <code>fetchPrepared</code> method for a description of <code>\$key</code>
<code>\$secondaryKey</code>	check <code>fetchPrepared</code> method for a description of <code>\$secondaryKey</code>

Returns:

Results set as described in `results` method.

method `results`

Does not take any parameters.

After executing a query (with either `fetch`, `insert`, `update` or `execute` method), this will return the results of the query execution. The results consist of an array that contains the details of the last operation. If an error occurred, the it will have the keys `errorMsg`, `errorCode`, `errorFile` and `errorLine` describing the nature of the error and the exact line that caused the error. Also under the key `error` there will be an HTML string that compiles the above information into a displayable message. Furthermore the key `query` will contain the query that caused the problem.

If no error occurred, the `error` key will be `FALSE`. In that case the results array will have the following information: `rows` will give the number of rows that were affected by the last query, `id` indicates the autoincrement id given to the last inserted row when doing an insertion and `data` will hold the actual payload returned from a `SELECT` statement (the format of which depends on which fetch method has been used).

method `update`

Updates the specified table. In order to guarantee security, a prepared statement is used for this operation.

Parameters:

<code>\$table</code>	the name of the table to update
<code>\$data</code>	an array of values to update, where the keys represent the fields in the table
<code>\$condition</code>	optional condition string, e.g. <code>id > 40 AND login = test</code> the condition string may also have '?' placeholders for prepared statements, but no named placeholders
<code>\$parameters</code>	if placeholders are used in the condition string, this array must carry the parameters

Returns:

Results set as described in `results` method.

method `insert`

Inserts an array of data in specified table. In order to guarantee security, a prepared statement is used for this operation.

Parameters:

<code>\$table</code>	the name of the table in which to insert
<code>\$data</code>	an array of rows to insert into the table; each row is an array where the keys represent the name of field in which the value is to be inserted

\$onDuplicateKey this tells rixPDO how to handle data that would conflict with already existing rows (based on a unique key). By default this is FALSE and doesn't do anything special (so there will be an error on a conflict). It can take 3 different values apart from FALSE:

- 'ignore' => conflicting data will be left out.
- 'replace' => existing data will be deleted and then replaced by the new one (this can be problematic when other tables with foreign keys relating to this row: CASCADE would delete all data that depends on this row, before the new row is written.
- 'update' => old data will be updated on conflict. This requires a list of columns to update (see next parameter).

\$updateKeys ignored unless the last parameter is set to 'update'. This is an array of strings containing column names that are to be updated in case of a conflict of unique keys.

Returns:

Results set as described in **results** method.

Notes:

- every row MUST have the same number of values and target identical keys, because the data will be inserted in a single SQL command
- if only a single row of data is to be inserted, the row is not required to be wrapped in an array with only this one row as an element; it can be given as is

method insertFile

Inserts binary data into a BLOB column of the database. This method only allows for one BLOB column per row, inserting multiple files in a single row is not supported.

Parameters:

\$table the name of the table to interact with

\$input a row of data to insert into the table, one value of which is the path to the file to insert (full server path, e.g. `$_FILES['file1']['tmp_name']` after uploading through a form)
like with the insert method, the keys of the row must match the names of the fields in the database table

\$fileColumn the key of the row that contains the path to the file

Returns:

Results set as described in **results** method.

method fetchFile

Fetches binary data from a BLOB field and sends it to the output buffer. Only a single field can be targeted with this method.

Parameters:

\$query the query that selects the data from the table
the query may contain placeholders in the WHERE clause

\$parameters the parameters to bind to the prepared statement, as an array
if the query was defined with named parameters, the keys of the array represent the names

`$headers` or `$returnflag` an array of strings that are to be sent as header before flushing the binary data to the output buffer; typical headers would include:

```
'Content-Description: File Transfer'
'Content-Type: application/octet-stream'
'Content-Disposition: attachment; filename="example.pdf"'
'Expires: 0'
'Cache-Control: must-revalidate'
'Content-Length: 16523'
```

if this argument is missing, no headers will be sent, and the data is flushed as is to the browser, which is usually not a good idea
if this argument is FALSE instead of an array, the binary data is returned as a string instead of written to the output buffer

Returns:

If headers are provided this returns a results set as described in `results` method. If `returnflag` is TRUE it returns the data from the file, instead. A results set is also returned if an error occurs.

method startTransaction

Starts a transaction. Does not take any parameters.

Returns:

Results set as described in `results` method in case of an error, otherwise it returns TRUE.

method rollback

Rolls back a transaction previously started with the `startTransaction` method. Does not take any parameters.

Returns:

Results set as described in `results` method in case of an error, otherwise it returns TRUE.

method commit

Commits a transaction previously started with the `startTransaction` method. Does not take any parameters.

Returns:

Results set as described in `results` method in case of an error, otherwise it returns TRUE.

method keyString

This creates a comma separated string with the keynames of an array. It is very convenient when creating an INSERT statement. Example:

```
INSERT INTO table_name (`firstName`,`lastName`,`title`) VALUES (?,?,:)
```

In this case the `keyString` method could create the `('firstName','lastName','title')` string from an array that has the 3 keys `firstName`, `lastName` and `title`.

Parameters:

<code>\$arr</code>	the array with the data, having the correct keys for the query
<code>\$backticks</code>	puts the keynames in backticks (true if omitted)
<code>\$parentheses</code>	adds brackets around the string (true if omitted)
<code>\$prefix</code>	prefix to add to every keyname (empty if omitted)
<code>\$postfix</code>	postfix to add to every keyname (empty if omitted)

Returns:

A string.

In the example above we might have the following code:

```
$data = ['firstName' => 'Alfred E.', 'lastName' => 'Neumann', 'title' => ''];  
$keyString = $db->keyString($data);
```

method variableString

This creates a comma separated string of placeholders for prepared statements. These can be either named placeholders or merely question marks. Example:

```
INSERT INTO table_name (`firstName`,`lastName`,`title`) VALUES (?,?,?)
```

In this case the variableString method could create the '(?,?,?)' string, or it could instead create a string of named placeholders like '(:firstName,:lastName,:title)'.

Parameters:

\$arr or \$n	the array with the data, having the correct keys for named placeholders instead of an array this can be an integer value with the count of question marks to create if not using named variables
\$parentheses	adds brackets around the string (true if omitted)

Returns:

A string.

In the example above we might have:

```
$varString = $db->variableString(3);
```

Or with named placeholders:

```
$data = ['firstName' => 'Alfred E.', 'lastName' => 'Neumann', 'title' => ''];  
$varString = $db->variableString($data);
```

method setAdditionalErrorData

This method can be executed before preparing a query if **errorHandling** mode 1 is being used. It allows the user to specify extra data to be returned together with the error message, should an error occur. This requires that **&\$errorVar** is an array and that **\$errorVarKey** is specified.

The data will be used until it's cleared manually with **clearAdditionalErrorData**.

Parameters:

\$data	an array with the extra data to send back; each key and its value will be copied to the \$errorVar specified at instantiation
--------	---

method clearAdditionalErrorData

This method clears additional error data previously set with the **setAdditionalErrorData** method.

Usage examples:

connection:

```
//database.php defines the variables $sql_db, $sql_user, $sql_password and $sql_host
require_once 'inc/php/database.php';
require_once 'inc/php/rixPDO.php';
$returnData['error'] = false;    //$returnData is an array returned in shutdown function

$db = new rixPDO($sql_db, $sql_user, $sql_password, $sql_host, '../logs/PDO_errors.txt', 1,
$returnData['error'], ['emulatePrepares' => true]);
```

fetching data:

```
$query = "SELECT * FROM log WHERE id > ?";
$parameters[] = 10;
$result = $db->fetchTable($query, $parameters);
```

fetching data with named placeholders:

```
$query = "SELECT * FROM log WHERE id > :id";
$parameters['id'] = 10;
$db->prepare($query);
$result = $db->fetchTable($query, $parameters);
```

executing a query:

```
$query = "INSERT INTO log (login, test) VALUES ('login1', 'test1')";
$db->execute($query);
$result = $db->results();
```

inserting multiple rows of data from an array:

```
$data = array();
for ($i = 1; $i<21; $i++) {
    $data[] = array('login' => 'login_.$i, 'test' => 'mainTest');
}
$result = $db->insert('log', $data);
```

updating a row of data from an array:

```
$data = array('serial' => 123456789, 'logpos' => 1);
$db->update('log', $data, 'id > 40');
$result = $db->results();
```

updating a row of data from an array with placeholders in condition string:

```
$data = array('serial' => 123456789, 'logpos' => 1);
$parameters = array(40);
$db->update('log', $data, 'id > ?', $parameters);
$result = $db->results();
```

executing a prepared statement:

```
$query = "INSERT INTO log (login, test) VALUES (?, ?)";
$db->prepare($query);
$parameters = array('login', 'test1');
$result = $db->executePrepared($parameters);
```